Tunza — Master Product Prompt for OpenCode (Sell-Ready, Multi-Tenant, Production-Grade)

Give this to OpenCode as the task brief and save it as `AGENTS.md`. Where it differs from earlier Tunza prompts (build prompt, remediation prompt), this one wins.

Prepared by Muhindo Vicent, Zest Technologies · Draft v1.0

0. Role and Operating Rules

You are a senior back-end engineer, a security-minded full-stack engineer and a DevOps-minded engineer in one. You are building **Tunza**, a maternal health platform for Uganda that Zest Technologies intends to **sell to organisations and government bodies**: NGOs, hospitals, district health offices and Ministry units. A pregnant woman is registered by a community health worker and receives week-by-week advice and reminders by SMS. Staff and partners use a private, role-based web platform. Pregnancy is special personal data under Uganda's Data Protection and Privacy Act, 2019, so security and governance are legal and commercial requirements.

1. **Plan first.** Print a short plan and wait for my confirmation. Record each major decision as an Architecture Decision Record in `docs/adr/`.
2. **Build in milestones** (Section 14). After each, run the tests, show results and stop. Do not continue until I say so.
3. **Do not invent.** Where this brief is silent, choose the safest option, log it in `DECISIONS.md` and continue. Ask only about irreversible choices.
4. **No real patient data, ever.** Use a synthetic data generator with obviously fake data (`0700 000 001`).
5. **Never weaken a security rule to make a test pass.** If a rule blocks you, tell me.
6. **Never log** phone numbers, names, message text tied to a person, passwords, tokens or OTP codes.
7. **Every feature ships with** tests, audit coverage, documentation and an entry in the OpenAPI spec.
8. **Prefer boring, well-maintained technology.** Justify every new dependency (licence, maintenance, size). Government buyers audit dependencies.

9. Keep `RUNBOOK.md`, `docs/threat-model.md` and `docs/architecture.md` current as you build, not at the end.

1. Product and Commercial Context

What Tunza is: an SMS companion for pregnant women plus a facility and community-health-worker management platform, aligned to the WHO and Uganda MOH 8-contact antenatal care (ANC) schedule.

What buyers care about: data sovereignty, auditability, uptime, support, interoperability with national systems, cost per woman, and proof that clinical content is governed.

Build once for three deployment models:

- **A. Hosted multi-tenant** run by Zest.
- **B. Dedicated single-tenant instance** per customer.
- **C. Customer-hosted**, delivered as containers on the customer's or government's infrastructure.

So: a tenant-aware schema, no hard dependency on any one cloud, all configuration through environment and tenant settings, and every service runnable from container images.

2. Honest Constraints (Read First)

1. **Code alone does not make this sellable.** A serious buyer will ask who approves the messages, who is liable if one is wrong, who answers a danger-sign alert at 2 a.m., and where the data lives. The platform therefore enforces **per-tenant go-live gates** (Section 15) and refuses to message real women until they are met. Never describe the system as cleared for real patients in the UI or the docs.
2. **Do not claim certifications** (ISO 27001, SOC 2 and similar). Build to be *ready for* them and say exactly that.
3. **Modular monolith, not microservices.** One deployable app with strict module boundaries is easier to secure, test, operate and hand over. Splitting services without a measured need is over-engineering. Record that decision in an ADR.
4. **Build by buyer evidence.** Sections 1 to 8 and 11 to 12 are the core. Build SSO, FHIR, Kubernetes and DHIS2 export only when a named buyer requires them, but keep the design ready for them.

5. **SMS cannot guarantee action.** The system notifies; people must act. The alert chain is a process agreed with the customer, not a software feature.

3. Architecture

```
 Internet -> Edge (nginx/Caddy: TLS, headers, rate limits, optional WAF
rules)
                |-- web (Flask + gunicorn, stateless, N replicas)
                |-- worker (outbox sender, planner, retention, reports,
webhooks)
                |-- PostgreSQL 16 (primary, optional replica) via PgBouncer
                |-- Object storage (S3-compatible): consent images, exports,
backups
                |-- Redis (optional): rate-limit counters, short caches
                |-- Observability: metrics, logs, traces, alerts
 SMS providers <-> /webhooks (delivery reports, inbound)   [provider
interface]
```

Modules (one package each, each owns its tables, no cross-module table access): `identity`, `tenancy`, `directory` (districts, parishes, facilities), `patients`, `consent`, `content`, `protocols`, `scheduling`, `messaging`, `inbound`, `alerts`, `care` (appointments, attendance), `reporting`, `integrations`, `admin`, `audit`, `platform` (metering, feature flags). Modules call each other only through service interfaces.

Capacity targets (assumptions to confirm with a buyer): 250,000 active women and 2 million SMS a month per large tenant; p95 API latency under 300 ms; planner run for 250,000 women under 5 minutes; 99.9 percent monthly availability; RPO 15 minutes; RTO 4 hours. Prove them with load tests.

4. Stack

Back end: Python 3.12, Flask 3, SQLAlchemy 2 Core, Alembic, Pydantic v2 validation, gunicorn, `argon2-cffi`, `cryptography`, `structlog`, OpenTelemetry. The OpenAPI 3.1 spec is generated from the code and published at `/api/v1/openapi.json`.

Database: PostgreSQL 16. Constraints, enums, CHECKs and foreign keys in the schema. SQLite is allowed only for a developer's laptop and fast unit tests, and CI runs the integration suite on real PostgreSQL (testcontainers).

Front end (hard constraint): vanilla HTML, CSS and JavaScript, with no framework and no build step. Make it modern anyway:

- ES modules, **Web Components (custom elements, prefixed `tz-`)** for the design system, native `<dialog>` and `popover`, CSS custom properties, cascade layers, container queries, `:has()`, View Transitions as progressive enhancement, `Intl` for dates, numbers and plurals.
- JSDoc types checked with `tsc --checkJs` as a dev-time check only, with no transpile step.
- No runtime CDN. Self-host all assets. A service worker caches **static assets only**, never API responses or personal data.
- **Performance budgets:** at most 100 KB gzipped JavaScript per page, LCP under 2.5 s on a mid-range Android over slow 4G, CLS under 0.1.
- **i18n:** JSON message catalogues, English and Luganda first, Kiswahili ready, no hard-coded strings.
- **Accessibility:** WCAG 2.2 AA, keyboard operable, visible focus, 44 px touch targets, status shown by icon plus text plus colour, works at 360 px width and 200 percent zoom.

Quality tooling: `pytest`, `hypothesis` (property tests for date logic), Playwright (end-to-end), k6 (load), `ruff`, `mypy` (strict on domain code), `bandit`, `pip-audit`, `gitleaks`, Trivy, Syft (SBOM), `cosign` (image signing).

Brand tokens: terracotta `#C1613C`, deep clay `#A8532F`, sage `#6F8D64`, light sage `#8FAE81`, warm brown `#4A3428`, cream `#FBF3E9`; success `#4B7A55`, warning `#C98A2D`, **danger `#B23A2E`**, info `#4E7A93`. Danger red must never be confused with the terracotta on ordinary buttons. Poppins for headings, regular body, warm and human tone. Tenants may override logo and primary colour through CSS variables, but danger and success colours are fixed.

5. Multi-Tenancy and Data Isolation

- `organisations` is the tenant table. **Every business table carries `org_id NOT NULL`.**
- **PostgreSQL Row-Level Security** on `org_id`, set per transaction with `SET LOCAL app.org_id`. The application database role has no `BYPASSRLS`, and migrations run under a separate role. RLS is defence in depth, **not a replacement** for application permission and scope checks.

- **Platform staff cannot read patient data by default.** Support access is time-limited, approved by the tenant, shows diagnostics only, and is fully audited. No impersonation.
- **Per-tenant settings:** logo and colours, enabled languages, care protocol version, alert SLAs, quiet hours, retention periods, sender IDs and SMS provider credentials (encrypted), quotas.
- **Per-tenant encryption keys:** envelope encryption, where each tenant's data key is wrapped by a master key from a KMS or secrets store. Offboarding a tenant can then crypto-shred its data after the retention period.
- **Tenant lifecycle:** onboarding wizard, suspend, full export in open formats (CSV and JSON), offboard.
- **Metering:** messages, segments and active women per tenant per month, as a usage report for billing. Billing itself is out of scope.

6. Identity, Roles and Authorisation (Authoritative)

Principles: individual accounts only; no one ever sees or sets another person's password; no self-assigned power; least privilege enforced on the server; separation of duties; everything audited.

Tier	Roles
Platform (Zest)	`platform_owner` (at least two), `platform_support` (no patient data)
Tenant	`org_admin`, `clinical_reviewer`, `dpo`, `supervisor` (district VHT coordinator), `vht`, `facility_staff`, `partner_viewer`

Who assigns roles (never by self-registration or by editing the database):

Role	Initiated by	Approved by
First `platform_owner`	Server CLI bootstrap (password prompted without echo, refuses to run if one exists)	None

Role	Initiated by	Approved by
More `platform_owner`, `org_admin` (first one per tenant), `dpo`	`platform_owner`	A second `platform_owner` and, for tenant roles, the customer's named signatory
`clinical_reviewer`	`org_admin`, with evidence	A `platform_owner` and an existing reviewer (the first reviewer: plus an uploaded verification record, flagged in audit)
`supervisor`, `vht`, `facility_staff`	`org_admin` (a `supervisor` may invite `vht` in their own district)	None extra, if the facility partnership is on file
`partner_viewer`	`org_admin`	A `platform_owner`; expires after 6 months unless renewed

No one grants authority equal to or above their own or approves their own request. A role or scope change ends that user's sessions. Users are deactivated, never deleted. The same person must not hold `dpo` and `org_admin`; warn if they do.

Credentials: an approved invitation creates a single-use random token (only its hash stored, 72-hour expiry, never logged). The invitee picks a username and their **own** password (12 or more characters, common-password check, Argon2id), verifies their phone by SMS code, accepts the confidentiality acknowledgement and completes a short orientation. Admins can trigger a reset link and never see any secret.

Authentication: identical error for unknown user and wrong password; lock for 15 minutes after 5 failures and notify the owner; rate limits by account and IP. **Second factor is mandatory** for every role except `partner_viewer`: SMS code as baseline, TOTP and WebAuthn passkeys supported and recommended for privileged roles. OIDC single sign-on is a later, buyer-driven option. The development console OTP path must be impossible when `APP_ENV=production`.

Sessions: server-side, cookie holds a random ID only (`HttpOnly`, `Secure`, `SameSite=Lax`), rotated on login and role change, CSRF token on every state-changing request, nothing sensitive in `localStorage`. Idle timeout 15 minutes for privileged roles

and 30 for the rest, absolute limit 12 hours, a warning two minutes before expiry, unsaved form drafts kept in page memory only, a list of active sessions with sign-out of other devices.

Authorisation: permission strings mapped to roles in one `permissions.py`, not editable from the UI. A decorator `@require_permission('x:y')` guards every route and fails closed, and a test fails if any route lacks it. **Scope is enforced in the SQL**: a VHT sees own women; facility staff see women whose home facility matches and who consented to facility sharing; partner views are aggregates with any cell under 5 suppressed. Out-of-scope records return 404 and are audited. Admins and platform roles cannot approve clinical content; the reviewer cannot manage users or configuration. Phone numbers are masked by default and a reveal is a logged click. Role pages are served only through gated Flask routes: `/public/*` needs no login; `/app/<role>/*` and `/api/v1/<role>/*` need the matching role and scope.

Audit: append-only `audit_log` (actor, role, tenant, action, object, patient, IP, user-agent hash, outcome, time), database triggers refusing `UPDATE` and `DELETE`, each row hash-chained to the previous, and shipped continuously to a separate store (object-lock bucket or external syslog) so a database admin cannot alter history silently. Log logins, failures, factors, lockouts, every patient read and write, phone reveals, role changes, template approvals, configuration changes, exports, go-live gate changes and every permission denial.

7. Clinical and Messaging Core

- **Care protocols:** versioned and reviewer-approved, per tenant. The default is the Uganda MOH 8-contact model (contacts at up to 12, 20, 26, 30, 34, 36, 38 and 40 weeks). Design a protocol as data so later packs (postnatal visits, infant immunisation) can be added without rewrites. Maternal tetanus dose tracking is in the core; infant immunisation is a later pack, off by default.
- **EDD** = LMP + 280 days; **gestational week** = floor((today - LMP) / 7); an ultrasound EDD override stores `edd_source`; reject a future LMP; warn beyond 42 weeks; use Africa/Kampala time; property-test boundaries.
- **Content rule:** message text comes only from approved, versioned templates and is never generated at send time. Templates seeded in development are marked `DRAFT - NOT CLINICALLY APPROVED` and can never be queued.

- **Planner:** a timed job turns the `message_plan` into `outbox` rows with a unique constraint on (patient, template), so running it twice is harmless. Eligibility at queue time and again at send time: active, SMS consent held and not withdrawn, template approved, language set, outside quiet hours (no non-emergency sends 9 pm to 6 am).
- **Worker:** leases rows (`FOR UPDATE SKIP LOCKED`), rechecks consent, sends, stores the provider message ID, writes `sent_log`. A crash can never double-send. Retries back off (1, 5, 15, 60 minutes, five attempts); permanent failures surface to the VHT and the tenant admin.
- **Provider interface:** `SmsGateway` with Africa's Talking and a console implementation first; support more than one provider per tenant with failover. **Check the current provider documentation** before coding callbacks and status codes. Do not assume callbacks are signed; use a secret path token, an IP allowlist where published, strict payload validation, rate limits and HTTPS only. Handle duplicate and out-of-order callbacks idempotently.
- **Encoding:** GSM-7 gives 160 characters (153 per part when joined), anything else switches to UCS-2 with 70 (67 per part). Show the live segment count in the reviewer's editor and block approval above a configurable maximum.
- **Inbound pipeline:** validate, normalise, then match in order: STOP (opt out at once, cancel pending sends, notify the VHT), danger signs, START, HELP, LANG, appointment reply, fallback. Danger categories: bleeding, severe headache or blurred vision, severe abdominal pain, reduced fetal movement, fever, swelling, convulsions, waters breaking. **When unsure, escalate.** Luganda keywords are placeholders until a native speaker and the clinical reviewer confirm them. Handle shared phones (never reveal pregnancy information to an unverified replier) and changed numbers.
- **Alerts:** default chain, configurable per tenant and agreed in writing with the customer: T+0 notify the VHT (SMS and dashboard), T+30 minutes notify the facility on call and the supervisor, T+60 minutes show an SLA breach to the tenant admin and send them an SMS.
- **Consent:** three scopes (`sms`, `vht_followup`, `facility_sharing`), checked before every action they govern. A woman under 18 cannot be saved without guardian details. Pause or stop for miscarriage, stillbirth, delivery or moving, with no further pregnancy messages.

8. Role Experiences (After Private Login)

Every role lands on **My day** with its three most urgent items. Every page has a Privacy and data link. Shared pages: login, first-login setup, reset, notifications, profile and security, help, and branded 403, 404 and session-expired pages. No role picker exists anywhere; the server decides the landing page.

Role	Does	Pages	Cannot
Platform owner	Approves privileged access, manages tenants, go-live gates, usage and platform health	Tenants, Approvals, Roles register, Security events, Platform health	See patients or passwords, approve clinical content
Org admin	Invites users, processes account requests, configures the tenant, watches delivery and cost, reviews audit	System health, Users, Requests, Configuration, Audit, Delivery and cost, Impact	Approve content, read patient records
Clinical reviewer	Approves templates and protocols, maintains danger keywords, reviews escalations, signs off languages and districts	Review queue, Message bank, Keywords with test box, Escalation log, Sign-offs	See identities, manage users
DPO	Consent oversight, data-subject requests, retention, breaches	Consent overview, Requests, Retention, Access anomalies, Breach log	Edit content, manage users
Supervisor	Oversees VHT workload and overdue lists, reassigns women, requests VHT accounts	Team workload, Overdue, Reassign	See outside own district

Role	Does	Pages	Cannot
VHT	Registers women, records consent, works caseload, handles alerts, updates status	My day, Register mother, Caseload, Patient, Alerts, Appointments	See others' women, edit content, bulk-export
Facility staff	Attendance forecast, confirm attended or missed, receive referrals	This week, Attendance, Referrals, Reference	See unlinked or unconsented women
Partner viewer	De-identified metrics, governance documents, suppressed reports	Overview, Governance, Reports	Retrieve any identifiable field

VHT field rules: no identifiable data in `localStorage` or IndexedDB. If the connection drops mid-form, keep the draft in page memory and retry. Show loading, empty, error and offline states in plain, warm language.

Public site: Home, How Tunza works, Clinical standard and data protection, For organisations, Request an account (rate-limited, spam-protected, feeds the admin queue), Contact. It describes roles but never shows a working dashboard or real data.

9. Interoperability and Reporting

- **Reports:** facility, district and programme aggregates, CSV and XLSX export with small-cell suppression, scheduled delivery to named recipients.
- **Outbound webhooks:** HMAC-signed, retried, SSRF-protected (deny private address ranges), events such as `patient.enrolled`, `alert.raised`, `alert.handled`.
- **API keys for integrations:** per tenant, scoped, hashed at rest, rotatable.
- **DHIS2:** de-identified aggregate push through a configurable mapping. Uganda's health system reports through DHIS2, but confirm the current MOH requirements before building.
- **FHIR R4:** a read-only facade (Patient, Appointment, Consent) behind a feature flag. Do not build until a buyer asks.

10. Security Programme

- Target **OWASP ASVS Level 2** and the OWASP Top 10. Keep a STRIDE threat model with data-flow diagrams in `docs/threat-model.md` and update it every milestone.
- Strict Content Security Policy without inline scripts, HSTS, frame denial, content-type and referrer headers, secure cookies, CSRF defence, rate limits on login, OTP, invitations, public forms and webhooks, request size limits, parameterised queries only, Pydantic validation and output escaping on every field.
- File uploads (consent form images): type sniffing, size limits, antivirus hook, object storage, never served inline from the app origin.
- Encryption: TLS 1.2 or higher (1.3 preferred), AES-256-GCM field-level encryption for names, phone numbers and guardian details with a key ID per value for rotation, HMAC `phone_hash` for lookups, disk and backup encryption.
- Least-privilege database roles, no public database port, admin pages restricted by IP or VPN option, secrets only from environment or a secrets manager, a committed secret fails CI.
- Supply chain: pinned dependencies, `pip-audit`, Trivy, SBOM per release, signed images, a vulnerability disclosure policy and `security.txt`.
- **Independent penetration test and fixes for every Critical and High finding before the first customer go-live.**
- Privacy engineering: data minimisation, retention jobs set by the DPO, data-subject request tooling, a DPIA template, a record-of-processing export and a breach register. Do not hard-code a breach notification deadline; confirm the legal period with counsel.

11. DevOps and Operations

- **Environments:** development, staging (production-like, synthetic data only), production.
- **Infrastructure as code:** Docker Compose for a single node with a documented scale-out path; Terraform or Ansible for provisioning; a Helm chart only when a buyer needs Kubernetes.
- **CI/CD:** lint, type check, tests on PostgreSQL, security scans, migration check, image build and signing, deploy only from a tagged commit, with manual approval for production.
- **Zero-downtime deploys:** expand and contract migrations, rolling or blue-green releases, feature flags, tested rollback.

- **Observability:** structured logs without personal data, metrics (rate, errors, duration, saturation), traces, dashboards, SLOs and alert rules. Metrics must include outbox depth, oldest unsent message, delivery rate, open alerts, oldest unacknowledged alert, provider balance and cost per tenant.
- **Dead-man's switch:** the scheduler and worker write heartbeats; silence for two intervals pages the on-call person by SMS and email. An external monitor checks `/readyz` (database, heartbeats, outbox).
- **Backups and disaster recovery:** daily full backup plus continuous WAL archiving for point-in-time recovery, off-site and encrypted, a monthly **restore drill**, documented RPO and RTO.
- **Runbooks:** provider outage, database failover, outbox stall, alert SLA breach, leaked credential, breach response, tenant offboarding, key rotation.
- **Cost controls:** per-tenant daily SMS spend caps and low-balance alerts.

12. Tests That Must Pass

Access, tenancy and governance

1. An anonymous request to any protected page or endpoint is redirected or returns 401.
2. Every role gets 403 for every other role's pages and endpoints; a route without a permission decorator fails the route-map test.
3. A user in tenant A can never read tenant B's data, tested through the API, direct SQL under the app role, and with RLS bypass attempts.
4. A VHT requesting another VHT's patient gets 404 and an audit entry.
5. Org admin and platform roles get 403 on template approval; the reviewer gets 403 on user and config endpoints.
6. Facility staff see no aggregate cell under 5 and no unlinked or unconsented woman; the partner viewer cannot retrieve any identifiable field.
7. Nobody approves their own request; privileged grants need two different people.
8. An invitation token works once, expires at 72 hours and is stored only as a hash; no endpoint returns a password or hash.
9. Five failed logins lock the account; a role change ends sessions; logout destroys the session; tampered or replayed cookies and requests without CSRF tokens are rejected.
10. `UPDATE` and `DELETE` on `audit_log` fail, and tampering with a chained row is detected.

Clinical and messaging

11. EDD and week calculations pass boundary, leap-year and invalid-LMP cases (including property tests).
12. A woman under 18 cannot be saved without guardian details.
13. Running the planner twice, or killing the worker mid-send, never double-sends.
14. A draft or unapproved template is never queued or sent.
15. An opted-out woman receives nothing further and pending rows are cancelled.
16. Duplicate and out-of-order delivery callbacks leave a consistent state.
17. Danger-sign keywords in both languages trigger the reply and the alert; segment counting is correct for GSM-7, UCS-2 and joined messages.
18. The platform refuses `live` messaging for a tenant while any go-live gate is open.

Operations and quality

19. Killing the worker triggers the dead-man's switch; a backup restores to a clean database and passes an integrity check.
20. The load test meets the Section 3 targets, and failure injection (provider timeout, database restart, replica loss) recovers without data loss.
21. Playwright role journeys pass for every role from a clean database, including keyboard-only runs and a 360 px layout.
22. Accessibility checks, performance budgets and security scans (ZAP baseline, dependency and image scans, secret scan) pass with no open Critical or High finding.
23. The seed command and any console OTP path refuse to run when `APP_ENV=production`.

13. Sell-Ready Deliverables (Procurement Pack)

- Architecture overview and diagrams, security white paper, threat model summary and a penetration test summary.
- Data protection pack: DPIA template, record of processing, consent form templates (English and Luganda, for legal review), retention policy, breach procedure.
- Hosting and disaster recovery description, SLA and support model templates, and an incident communication plan.
- OpenAPI documentation and an integration guide.
- Admin guide, one-page quick-start card per role, a training curriculum for VHTs and facility staff.

- A demo environment with synthetic data and a scripted demonstration for each role.
- A pilot plan with an evaluation framework and an onboarding checklist for a new organisation.
- A dependency licence inventory (SBOM), release notes and a documented release process.

14. Milestones (Stop After Each)

1. **M0 Foundations:** repo layout, ADRs, CI, schema and migrations, provider interface, design tokens.
2. **M1 Tenancy and identity:** organisations, bootstrap, invitations, approvals, first login, factors, lockout, sessions.
3. **M2 Authorisation:** permissions, scoped query layer, RLS, route-map test, hash-chained audit.
4. **M3 Directory, patients and consent:** facilities, registration, guardians, consent scopes, encryption.
5. **M4 Content and protocols:** templates, versioning, reviewer approval, keywords, care protocols.
6. **M5 Scheduling and delivery:** planner, outbox, worker, gateways, delivery reports, cost tracking.
7. **M6 Inbound and alerts:** webhooks, conversation state, STOP, danger signs, escalation timers.
8. **M7 Role experiences:** design system as Web Components, public site, then every role's pages.
9. **M8 Reporting and integrations:** reports, exports, webhooks, API keys.
10. **M9 Observability and DevOps:** health endpoints, heartbeats, dashboards, Compose and IaC, backups and restore drill.
11. **M10 Security hardening:** threat model review, scans, headers, ZAP, penetration-test readiness.
12. **M11 Performance and recovery:** load tests, failure injection, DR exercise.
13. **M12 Sell-ready pack:** documentation, demo environment, go-live gates report.

15. Go-Live Gates per Tenant (Not Solvable by Code)

Each gate is a row in `go_live_gates` with evidence and two approvers.

1. A named clinical reviewer has approved every template in every language, and a native speaker checked the Luganda.
2. The consent form and the minors' pathway are legally reviewed.
3. PDPO data-controller registration is filed and a Data Protection Officer is appointed.
4. A DPIA and the hosting-location decision are recorded.
5. SMS provider production account, Sender ID and two-way shortcode are approved.
6. The alert-response chain is agreed in writing with the facility and supervisor.
7. A restore drill has succeeded and the incident plan is documented.
8. The independent security review is done and findings are resolved.
9. Staff are trained, and the pilot is limited to a small, known, consented group in one district.
10. A support and SLA arrangement is signed with the customer.

16. Decisions to Flag, Not Assume

- Hosted multi-tenant, dedicated instance or customer-hosted for the first buyer, and where the data may legally and contractually reside.
- Whether facilities may enrol women directly. This brief assumes only VHTs enrol.
- Who becomes each tenant's first clinical reviewer and DPO.
- Whether the buyer needs SSO, DHIS2 or FHIR in the first release.
- Alert SLAs and escalation contacts.
- Pricing metrics (per woman, per message, per facility), so the metering hooks match.